Group_06_Bernacchia_Fernandez

# WEB SERVICE - Task Manager

## Abstract

This report explores the design and implementation of a web service dedicated to task management, inspired by tools such as *Trello* and *GitLab*. The motivation stems from the increasing need for ***coordination and clarity in project workflows***, especially within teams. After defining the problem and analyzing the existing solutions (*state of the art*), we present our technical approach, built around a **SQLite database**, **Flask backend**, and a **Python client library**. The project demonstrates a functional and user-friendly API with statistical insights, offering a foundational solution to distributed task tracking and management. The resulting architecture promotes **modularity, testability, and ease of use**, and provides a solid starting point for further development in real-world collaborative contexts.

## Introduction

*mettere link alla sezione corrispondente*
This report is structured in several chapters, each addressing a different aspect of the project development process:

1. **Motivation and Context**

   *explains the relevance of task management tools in modern work environments*
2. **Problem**

   *defines the problem tackled and its key aspects*
3. **State of the Art**

   *provides an overview of existing solutions and how our work relates to them*
4. **Appoach to the problem**

   *details our methodology, from data storage to API implementation*

   1. data storage - SQLite

   2. backend - Flask App

   3. API - Python client

   4. distribution - Wheel library
5. **Results**

   *presents the outcomes of our implementation and testing (including a small demo app)*
6. **Future Improvements**

   *suggests potential enhancements to extend functionality, usability, and scalability of the system*
7. **Conclusion**

   *summarizes contribution, scope, and limitations of the work in relation to the state of the art*

## 1. Motivation and Context

Cooperation and coordination are essential ingredients for team success. While small teams can easily manage tasks informally, as projects grow in size and complexity, the need for structured task tracking tools becomes critical. In this context, a simple, fast, and user-friendly web service can greatly improve project transparency and individual accountability.

Our goal was to create such a tool: a backend service capable of storing task data in a SQLite database, and providing endpoints for users to retrieve, update, and manage this data efficiently. To enrich the user experience, we also integrated a set of basic statistical features, available through a Python API.

# 2. Problem

This project targets any collaborative environment in which a project can be divided into individual tasks. More specifically, it is structured around the @JohnVans123/ProjectManagementDataset, which models a variety of projects, each composed of multiple tasks assigned to users.

Each Task is represented as a row in a table, with fields such as:

| Task ID | Task Name | Task Status | Assigned To | Hours spent | Budget | Actual Cost | Progress | Location | Start Date | End Date | Project ID | Project Name | Project Type |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |

We adapted this dataset to model a collaborative system where multiple users can take part in projects (instead of only one), assign themselves tasks, and update task statuses. This framework can also serve as a personal or professional project history, supporting performance tracking and resume building. Leaving always the team up to date to the modification applied and the progress gained.

# 3. State of the Art

This is primarily a **didactic project**, aimed at *applying and consolidating the knowledge* acquired in the course. One requirement was to use GitLab, which also offers built-in task planning tools. Inspired by **GitLab's Plan** section and by **Trello**, a popular task management platform, we aimed to replicate a minimal, extensible version of such tools.
Although our implementation is far less complex than Trello, we restructured the dataset to create a functional and coherent service that supports data access, team coordination, and progress visualization.

# 4. Approach to the problem

First, we divided the web service creation into manageable issues (tasks) and designed specific solutions for each component:

1. data storage → **SQLite database**
2. backend → **Flask app with CRUD endpoints**
3. API → **object-oriented Python client**
4. distribution → `wheel` **library permitting the usage of the api**

## 4.1 data storage - SQLite database

The first issue to address was where to store the data provided by the user. Since this is a university project, unlikely to require high scalability or heavy concurrent access, we opted for **SQLite** as our database.

> ***SQLite*** is a lightweight, serverless database engine widely used for prototyping and small-scale applications.

SQLite is natively supported in Python and has a straightforward setup, making it ideal for rapid development and easy integration with our web service. It doesn't require a separate server process, and all data is stored in a single file, which makes it very convenient for local testing and development.

Given that the project required exposing data through an API, we also decided to use **Flask**, the Python web micro-framework.

> ***Flask*** is a Python micro-framework ideal for building web services and REST APIs.

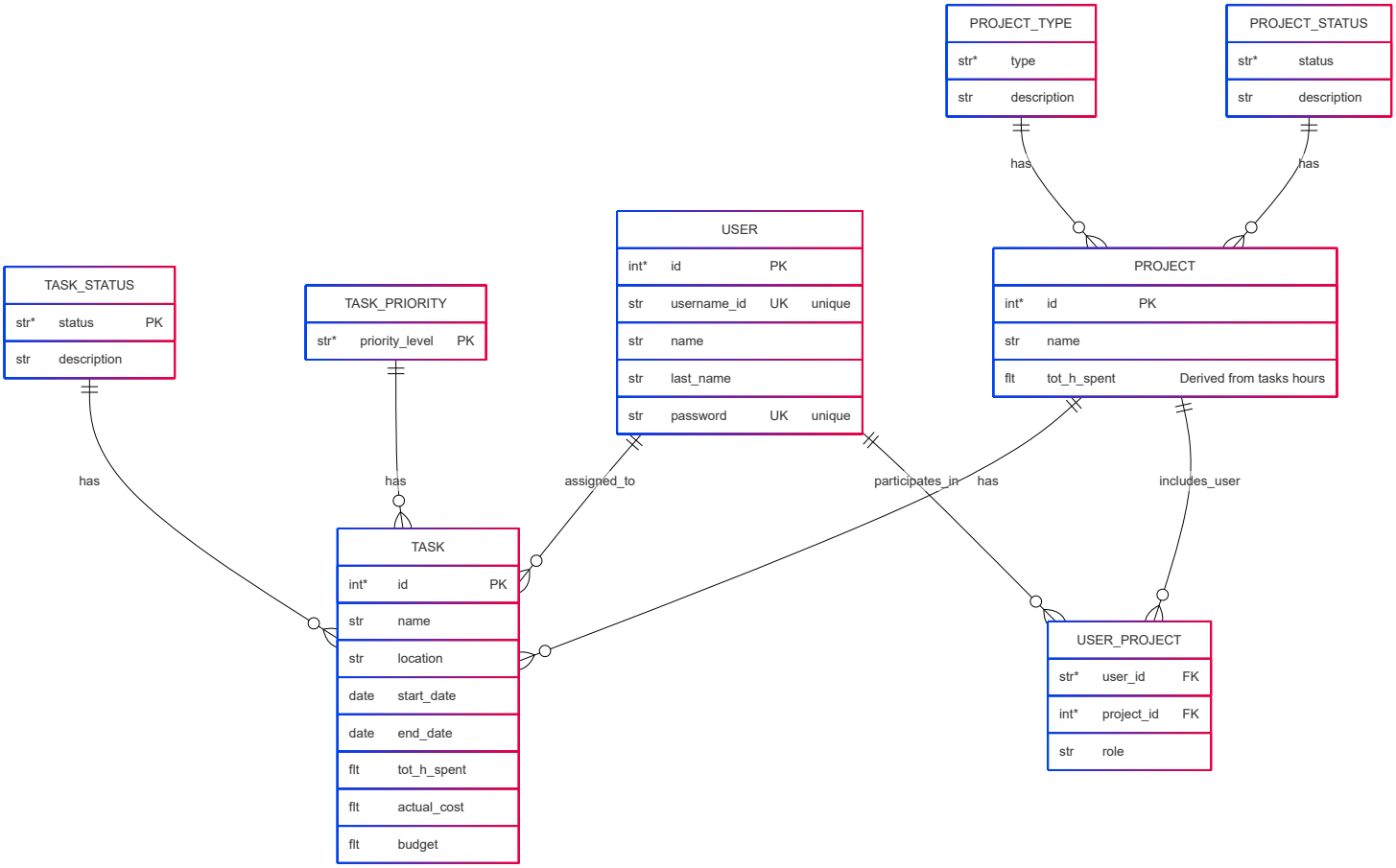
Flask is lightweight, well-documented, and integrates smoothly with SQLite and SQLAlchemy. Its simplicity allows us to focus on the core functionality of the web service while keeping the codebase minimal and readable. For our needs (CRUD operations, relational data exposure, and simple statistics) Flask is a pragmatic and efficient solution.

We adapted and extended the HuggingFace dataset, normalizing it into multiple tables to improve data management and enforce logical relationships. For example, we manually added user `roles` per project and allowed multiple `users` per `task`, resulting in a more realistic and flexible data model.

| **We defined three strong entities** | | **with two many-to-many relationships** |
|---|---|---|
| **User** | | **User ⇄ Project** |
| **Project** | | **User ⇄ Task** |
| **Task** | | |

Each many-to-many relationship was modeled as a separate association table. Every table has a primary key and one or more foreign keys, typically referencing fixed options or pre-defined categories (e.g., user roles).

The resulting Entity-Relationship Diagram is shown below:




Each table was mapped to a SQLAlchemy **model** to manage integrity constraints and enable object-relational mapping (*ORM*). This approach allows us to interact with the database using *Python classes* and ensures consistent data structure and validation across the application.

The HuggingFace dataset was used to populate the backend from the start, enabling iterative testing and validation of how the application behaves in a realistic context.

## 4.2 requests and backend - Flask App

After defining the **models**, we passed them to a dedicated **service** layer, which *handles all the queries to the database*. This layer implements the full set of **CRUD operations** (Create, Read, Update, and Delete) for each entity. It serves as a clean interface between the database and the application logic, ensuring that all interactions with the data are handled in a *consistent* and *maintainable* way.

All the services are then used inside the **Flask App's routes**, which expose *API endpoints* for external access via *HTTP requests*.

This separation of concerns allows us to modularize the backend:

- **routes** handle the request logic
- **services** handle data access
- **models** enforce structure and constraints

Using Flask also made it easy to continuously test the API during development. We relied heavily on **Postman**, a collaboration platform for API development and testing.

> ***Postman*** allows developers to test API endpoints through a user-friendly interface, automate request sequences, verify response correctness, and debug errors efficiently.

As a result, we developed a fully functional **backend** capable of retrieving and manipulating structured data through a clean and extensible **REST API**.

> ***RE****presentational* ***S****tate* ***T****ransfer* ***API*** is an architectural style for designing networked applications.
> It relies on standard HTTP methods (GET, POST, PUT, DELETE) to interact with resources identified by URLs.
> REST APIs are stateless, scalable, and language-independent, making them ideal for web services and client-server communication.

To ensure that the entire system worked correctly under different scenarios, we conducted **interaction testing** using the **pytest** framework.

> ***interaction testing*** ensures components (e.g. services, models, and routes) interact correctly under various conditions and that application logic behaves as expected across modules.

> ***Pytest*** is a powerful testing framework for Python, known for its simplicity and flexibility.

We implemented **125 tests**, covering:

- *input validation*

  e.g. field types, keys ignored during posts, primary keys inserted...
- *error handling*

  e.g. non-existent user IDs, non-existent relationship, invalid field for defined foreign keys...
- *consistency of changes across services*

Each test ensures correct request behavior and proper database updates or error returns.

## 4.3 API - python client

This component represents the actual product designed for the client: *a package that provides all the necessary functions by interacting with the backend through its routes*.
The **client-side API** exposes all the functionalities required to *fetch, update, manipulate*, and *delete* data from the database.

The key difference between the client and the backend lies in the implementation style: the client is ***fully object-oriented***. This means that the user can interact with the database entirely through **Python classes and functions**, without having to deal directly with HTTP requests or low-level backend logic.

The client library is structured into **modules**, each of which contains *Python classes* representing the core entities: **User, Project**, and **Task**. These mirror the backend models and are designed to be initialized and used directly when interacting with the API.

Each class provides:

- a **class method** that converts a *JSON response* from the backend into a proper *Python object* (handling default or missing fields gracefully),
- an *instance method* that does the reverse: converts a *class instance* into a *dictionary* suitable for serialization and use as the body of a *POST or PUT HTTP request*.

A key design principle is that these classes are *"dumb"*: they contain no embedded logic for HTTP communication. Instead, all communication is handled by the API layer, making the architecture *clean and modular*.

The API client includes four core types of methods:

- `GET` : these functions either return a **full list of objects of a given type** (with no arguments) or retrieve a **specific object** using its id. Under the hood, these make *GET requests* to the appropriate backend route.

- `CREATE` : these functions accept an object instance and use a *POST request* to **create the corresponding entity in the database**. The backend returns the **created object**, now including its assigned id.
- `UPDATE` : these require an object with a valid id. They make a *PUT request* to **update the corresponding entity in the database** with the new data.
- `DELETE` : these take an id and **remove the corresponding entity from the database**.

### Relationship Handling

Many-to-many relationships (e.g. between `User` and `Project` , or `User` and `Task` ) are handled slightly differently. These associations are *"silent"*, they cannot be retrieved directly via a dedicated endpoint, but they are visible indirectly:
*For example, when requesting all projects of a user, or all users of a project, the role attribute in the relationship is included in the result.*

To create or modify these relationships, a *POST request* is made with both involved `entities` and the `role` . If the relationship already exists, it is **updated**; if not, it is **created**.

### Response Integrity & Feedback

Since the `entity classes` themselves are decoupled from the live state of the database, **inconsistencies** could occur. To prevent this, every API `response` is passed through the `terminal_response` function.

This function:

- prints a **success message** to the terminal and continues execution if the operation succeeded,
- raises an **exception** and interrupt the program execution if an error is returned by the backend,
- prints a **warning** but continues execution in case of non-fatal issues (e.g., ignored keys in an update request).

This mechanism ensures **data consistency** and immediate **feedback** during usage.

### Unit Tests

The client module was tested with **22 class-level unit tests**, validating that:

- `constructors` work correctly with both full and partial parameters,
- `JSON-to-object` and `object-to-dictionary` conversions behave as expected.

Although **mock tests** would have been appropriate to simulate backend interactions, due to time constraints and their non-mandatory nature for the deliverables, they were deferred and listed among the possible future improvements.

> ***Mock tests*** simulate external dependencies (e.g., APIs) using mock objects to enable isolated testing.

### Additional Modules

To further enrich the client capabilities, three additional modules were developed:

- `aggregations.py` : provides aggregated data such as the *overall progress of a project* (as percentage of completed tasks) or the *total hours a user has worked on a project* or in *general*.
- `statistics.py` : contains *visualizations* to offer high-level insights into the database contents; such as: *task status distributions within a project*, *each user's contribution in percentage* and a *Gantt chart of a specific project's tasks*.
- `interface.py` : offers *simple textual overviews* of data related to specific entities, improving the readability of API outputs.

## 4.4 distribution - wheel package

The project has been finalized as a distributable **Python library** using the **wheel packaging system**, as introduced during the course.

> ***Wheel*** is a built-package format for Python, intended to speed up installations and make package management more reliable.

This approach was ideal in our case, as it allowed us to **isolate and export** only the `task_manager_library` folder, making it easily installable and reusable in other Python projects.

To demonstrate and validate the functionalities of our library, we also developed a small **demo application** named `demo.py`. This script showcases a *minimal working example using the client API*.

In the current implementation, our backend runs with a *pre-filled database* using the HuggingFace dataset. However, in a real-world deployment, the system would typically start with an *empty database* that the user populates over time.
For demonstration purposes, having pre-populated data is essential to effectively showcase the API's capabilities.

# 5. Results

The final product consists of a **fully functional and tested task management system**, composed of:

- a **Flask-based REST API**, capable of handling ***CRUD operations*** across all key entities ( `Users` , `Projects` , `Tasks` ), with consistent validation and robust error handling.
- A **SQLite database** organized in a ***relational schema with many-to-many relationships***, enhanced with foreign keys, constraints, and realistic normalization.
- A **Python client library**, fully ***object-oriented***, offering a high-level abstraction over the backend logic and enabling seamless interaction with the API.
- A suite of about **150 automated tests** using `pytest` , ensuring correctness of functionality, interaction consistency, and robustness under a wide range of input scenarios.
- A sample **demo script**, illustrating basic workflows such as project creation, user-task assignment, and status updates.

We validated the system by simulating typical user interactions, including:

- Creating users and assigning them to multiple projects.
- Defining tasks with deadlines and costs.
- Tracking project progress and user workload.
- Extracting basic statistics, such as completion rates and total budget spent per project.

The results confirm that the application is **capable of supporting basic task management workflows efficiently and intuitively**.
The architecture also supports future extensions without major restructuring.

# 6. Future improvements

Due to ***time constraints***, the project was not developed as thoroughly as we would have been capable of. Several functionalities that could have enriched and completed the application are currently missing.

- **Additional Aggregations and Statistics**: we did not have enough time to design and implement further aggregations or statistics to visualize. It would have been interesting to develop some predictive models based on user-provided data—for instance, providing personalized predictions on the estimated time a specific user might need to complete a task, or similar insights.
- **Enhanced Visualization and Dashboards**: as for the statistics and interface, it would have been valuable to implement dashboards for each entity type. This would allow combining data visualizations with interface panels displaying entity-specific information. At present, the related functionalities are limited to `interface.py` and could be extended and integrated more comprehensively into the overall client.
- **Complete Testing Suite**: from a development and software engineering perspective, completing the testing suite would be essential for a real-world or long-term deployment.
  In particular, the project would benefit from:
  - ***Unit tests for backend services***,
  - ***Interaction tests for the Python client API***,
    possibly including mock databases to simulate and validate endpoint behavior.
- **Scalability and Database Optimization**: lastly, if the application were to scale to a larger user base and data volume, the current service and database (**SQLite**) may not be optimal. During the implementation of certain statistical functions, we noticed that **SQLite** can be quite slow for more complex operations over larger datasets. In such a scenario, it would be worth considering switching to a more performant database system and updating the `service` logic accordingly.

# 7. Conclusions

This project delivered a **modular and extensible web service for task management**, tailored for team collaboration and progress monitoring. The use of **SQLite**, **Flask**, and an object-oriented **Python client** allowed us to focus on clarity, maintainability, and testability throughout the development process.

Compared to existing tools such as **Trello** and **GitLab's Plan section**, our solution is deliberately **minimal**, but it captures the essential mechanics of collaborative task assignment and tracking. The separation between backend logic, database, and client API encourages clean architecture and simplifies further development or deployment.

While the current implementation is ***suitable for academic or small-team contexts***, future improvements (e.g., user authentication, UI integration, advanced analytics) could bring the system closer to production-level quality.

Ultimately, the project highlights the value of structured planning, modular design, and testing in the development of even small-scale software systems. It serves as a concrete application of the software engineering principles studied in the course and a foundation for future exploration.